

这个项目是什么？

想象你是一位乡镇干部，想要把你们村打造成研学旅行基地。但从零开始写方案、设计海报、做宣传物料...这些事情做下来可能要花好几个星期。

这个工具能一键帮你搞定这一切。



一句话描述：输入村庄名称，它会自动分析村庄特色，然后生成一整套研学旅行解决方案，包括 PDF 文档、课程教案、宣传海报、品牌 Logo 等。

你能得到什么？



研学解决方案 PDF

完整的研学旅行方案，包含日程安排、预算、安全预案等



九宫格寻宝卡

有趣的互动探索卡片，学生在村里找宝藏



5份课程教案

详细的教学设计，每份都包含目标、准备、实施步骤



夏/冬令营手册

完整的营地生活指南



宣传海报

9张精美的研学活动海报



品牌视觉

Logo 设计、品牌标语、文创产品图



看看它长什么样

息，右边显示生成结果。

怎么用它？

整个使用流程只有三步：

1

填写村庄信息

输入村名、省份、城市、县区。还可以上传村庄图片或地方文献（如县志），让 AI 更了解这个村。

2

点击「开始生成」

系统会调用 AI 分析图片、理解地方特色，然后生成各种内容。这个过程需要几分钟。

3

下载生成的文件

所有 PDF 和图片都会显示在页面上，点击即可下载。



需要配置 API Key：这个工具依赖 AI 服务，需要配置 MiniMax 的 API Key 才能工作。API Key 就像一把钥匙，没有它就无法调用 AI。

技术架构一览

用户界面

Gradio 网页

↓

核心引擎

AI Agent 集群

↓

生成器

PDF + 图片

↓

输出

研学方案文件

✖ 这个项目由哪些部分组成？

项目代码组织成几个主要目录：

```
village_study_tour/  
agents/ - AI 智能体，负责生成各种内容  
solution_agent.py  
lesson_plan_agent.py  
camp_manual_agent.py  
exhibition_agent.py  
curriculum_agent.py  
generators/ - 将内容变成 PDF/图片  
pdf_solution.py  
pdf_lesson_plan.py  
poster_generator.py  
brand_generator.py  
models/ - 数据结构定义  
knowledge/ - 知识库  
rag/ - RAG 增强搜索  
app.py - 网页界面入口  
output_coordinator.py - 流程编排器
```



关键理解：每个 `xxx_agent.py` 就是一个“专家”，专门负责生成一类内容。比如 `solution_agent.py` 负责生成研学解决方案。

这些 Agent 怎么工作的？

每个 Agent 的工作原理类似：

1. 接收村庄信息
2. 构建提示词（Prompt）
3. 调用 AI 接口

[CODE](#)

乡村文旅研学生成器

PLAIN ENGLISH

告诉它要处理哪个村庄

写一封详细的"任务书", 告诉 AI 要做什么

把任务书发给 AI 服务 (MiniMax)

AI 返回的是 JSON 格式的文本, 需要提取出来

把结果交给下一步处理

如果想要修改「课程教案」的内容生成逻辑, 应该修改哪个文件?

☐ A. app.py

☐ B. agents/lesson_plan_agent.py

☐ C. generators/pdf_lesson_plan.py

检查答案

重新尝试

认识这些 AI Agent

项目的"大脑"是由多个 AI Agent 组成的团队, 每个 Agent 专攻一个领域。



我有龙潭村的信息, 帮我生成研学方案吧!



目标



我是 LessonPlanAgent，我来设计 5 门课程的具体教案！



我是 CampManualAgent，我来写夏令营和冬令营的完整手册！



我是 ExhibitionAgent，我来设计策展方案！



我是 CurriculumAgent，我来构建完整的课程体系！



Agent 一览表

Agent	负责生成	输出格式
SolutionAgent	研学旅行解决方案	JSON（含日程、预算、安全预案）
LessonPlanAgent	课程教案	JSON（含目标、准备、活动步骤）
CampManualAgent	营地手册	JSON（含日程、生活指南、安全）
ExhibitionAgent	策展方案	JSON（含展区、动线、装置）
CurriculumAgent	课程体系	JSON（含理念、方法、序列）



Agent 是怎么工作的？

让我们深入看看一个 Agent 的内部结构。以 SolutionAgent 为例：

乡村文旅研学生成器

每个 Agent 都有一个"系统提示词", 告诉 AI 它扮演什么角色、应该怎么思考、输出什么格式。这是保证 AI 输出质量的关键。

System Prompt 里面写了什么?

CODE

你是一个专注于中国乡村文旅
研学方案设计的专家AI...

- ## 核心方法论
- 1. 县域格局分析
 - 2. 人文地理纵深
 - 3. 研学价值定位

- ## 固定研学活动
- 每个方案必须包含:
- 村史馆参观
 - 九宫格寻宝卡

输出格式

严格按JSON输出...

PLAIN ENGLISH

告诉 AI: 你是乡村文旅专家

给 AI 分析问题的框架

规定必须包含哪些内容

要求必须输出 JSON 格式

Agent 的工作流程

败就返回一个默认模板。

System Prompt 在 **Agent** 中起什么作用？

- ☐ A. 显示在网页界面上给用户看
- ☐ B. 存储在数据库中
- ☐ C. 告诉 AI 怎么扮演角色和输出格式

检查答案

重新尝试



生成器：内容的最后一步

Agent 生成的是 JSON 数据，但用户需要的是 PDF 和图片。这就需要"生成器"来转换。



简单理解：Agent 负责"想内容"，Generator 负责"做文件"。

生成器类型



PDF 生成器

将 JSON 数据渲染成 PDF 文档。支持多种模板：解决方案、教案、营地手册等。

generators/pdf_*.py



海报生成器

根据内容生成 AI 图片提示词，然后调用图片生成服务。

乡村文旅研学生成器

品牌生成器

生成 Logo、品牌标语、文创产品图。

`generators/brand_generator.py`



寻宝卡生成器

生成九宫格寻宝卡（HTML/PDF）。

`generators/treasure_hunt_generator.py`

PDF 是怎么生成的？

PDF 生成经历了两个阶段：

1

JSON → HTML

用 Python 的字符串模板，将 JSON 数据嵌入到 HTML 模板中，生成带样式的 HTML。

2

HTML → PDF

用 **WeasyPrint**（或 ReportLab 备选）将 HTML 转换成 PDF 文件。



为什么用 HTML 作为中间格式？

HTML + CSS 可以轻松实现复杂的排版效果，而纯代码生成 PDF 则困难得多。先用熟悉的 Web 技术做排版，再转成 PDF，是很聪明的做法！



数据是怎么流动的？

当你点击「开始生成」后，程序内部发生了一系列精心编排的步骤。



幕后英雄：OutputCoordinator

这个文件（`output_coordinator.py`）就像一位导演，负责协调所有 Agent 和 Generator 按顺序工作。



实际代码中的流程控制

CODE

```
# output_coordinator.py
def run_full_pipeline(self, village):
    # 1. 提取村庄特色
    features = self.image_analyzer.analyze_images(...)

    # 2. 生成研学方案
    solution = self.solution_agent.generate(...)
    generate_solution_pdf(solution, ...)

    # 3. 生成教案
    lesson_plans = self.lesson_plan_agent.generate_all(...)
    for plan in lesson_plans:
        generate_lesson_plan_pdf(plan, ...)

    # ... 更多步骤
    return results
```

PLAIN ENGLISH

这是整个流程的入口函数

先分析图片，了解这个村有什么特色

让方案 Agent 生成研学旅行方案，并输出 PDF

让教案 Agent 生成多份教案，每份都输出 PDF

最后返回所有生成的文件路径

🔍 数据是怎么传递的？

每个 Agent 接收什么输入，又产出什么输出？让我们看一个具体的例子：

SolutionAgent 的输入与输出

📥 输入

- **village** — 村庄基本信息（名称、地址等）
- **features** — 村庄特色（地理、历史、非遗等）
- **local_documents_info** — 地方文献信息（可选）

📤 输出

- **title** — 方案标题
- **overview** — 概述（主题、受众、时长）
- **course_design** — 每日课程安排
- **logistics** — 后勤保障
- **budget** — 预算
- **treasure_hunt** — 寻宝卡数据



关键洞察：Agent 之间的数据传递是**链式**的。前一个 Agent 的输出，可能成为后一个 Agent 的输入。比如 SolutionAgent 生成的 `treasure_hunt` 数据，会被用来生成寻宝卡 PDF。

数据传递的"接力赛"

LessonPlanAgent（教案 Agent）需要哪些输入数据？

- ☐ A. 只需要村庄名称
- ☐ B. VillageFeatures（村庄特色）

检查答案

重新尝试



PDF 生成原理

用户最终需要的是可以下载的 PDF 文件。那么 JSON 数据是怎么变成漂亮的 PDF 的？



核心转换流程：

JSON 数据 → HTML 模板 → WeasyPrint → PDF 文件

第一步：JSON 转 HTML

用 Python 字符串拼接，把数据嵌入 HTML 模板：

CODE

```
html = ""
html += ""
html += "
" + solution.title + "
"

html += "
主题：" + overview.theme + "
"

html += "..."
```

PLAIN ENGLISH

开始构建 HTML 文档

添加 CSS 样式（字体、颜色、间距）

乡村文旅研学生成器

继续插入更多内容...

第二步：HTML 转 PDF

使用 WeasyPrint 库进行转换：

CODE

```
from weasyprint import HTML

# 将 HTML 字符串转换为 PDF
HTML(string=html).write_pdf(output_path)
```

PLAIN ENGLISH

从 WeasyPrint 库导入 HTML 类

告诉它"把这个 HTML 转成 PDF，保存到这个路径"

 PDF 模板设计

每种文档类型都有专门的模板，定义了不同的布局和样式。

模板类型



研学解决方案

封面 + 村庄概述 + 课程安排表格 + 预算表格

pdf_solution.py



课程教案

目标 + 准备清单 + 活动步骤表格

pdf_lesson_plan.py

乡村文旅研学生成器

日程表 + 生活指南 + 安全须知

pdf_camp_manual.py



策展方案

展区卡片 + 动线图 + 装置清单

pdf_exhibition.py



课程体系

理念 + 教学方法 + 课程序列表

pdf_curriculum.py

CSS 样式示例

CODE

```
body {
  font-family: SimSun; /* 宋体 */
  margin: 2cm;          /* 页边距 */
  color: #333;          /* 正文颜色 */
}
h1 {
  color: #2D5016;       /* 深绿色 */
  text-align: center;
}
table {
  width: 100%;
  border-collapse: collapse;
}
```

PLAIN ENGLISH

使用宋体（系统自带中文字体）

页面四周留 2 厘米空白

标题用深绿色，居中显示

表格边框合并成单线

如果 WeasyPrint 不可用（缺少 GTK3 库），系统会自动切换到 ReportLab 生成 PDF。ReportLab 是纯 Python 的 PDF 库，但需要更多代码来控制样式。



生成器的封装

每个 PDF 生成器都遵循统一的接口设计，方便调用：

CODE

```
def generate_solution_pdf(solution, features, output_path):
    """
    生成研学解决方案 PDF

    参数:
        solution: SolutionAgent 返回的数据
        features: 村庄特色信息
        output_path: 输出文件路径
    """

    # 1. 提取数据
    ov = solution.get("overview", {})
    bg = solution.get("background", {})
    courses = solution.get("course_design", [])

    # 2. 构建 HTML
    html = build_html(solution, features)

    # 3. 转换为 PDF
    HTML(string=html).write_pdf(output_path)
```

PLAIN ENGLISH

函数名清晰说明用途

文档注释解释每个参数

乡村文旅研学生成器

用模板构建 HTML

调用 WeasyPrint 输出 PDF

调用方式

CODE

```
# 在 OutputCoordinator 中调用
solution = solution_agent.generate(...)
solution_path = village_dir / "研学方案.pdf"
generate_solution_pdf(solution, features, str(solution_path))
```

PLAIN ENGLISH

先让 Agent 生成方案数据

指定输出文件路径

一行代码生成 PDF!

为什么有时 **WeasyPrint** 无法工作，需要切换到 **ReportLab**？

☐ A. 缺少 GTK3 运行时库

☐ B. WeasyPrint 版本太旧

☐ C. Python 版本不兼容

检查答案

重新尝试

图片是怎么生成的？

除了 PDF，用户还需要精美的海报、Logo 等图片。这些是怎么生成的？



图片生成的秘密：先用 AI 生成“图片提示词”（Prompt），再用图片生成服务（如 Lovart、MiniMax）根据提示词生成图片。

两步走策略

1

生成提示词

根据内容生成详细的图片描述，包括风格、元素、构图等。

generators/poster_generator.py

2

调用图片服务

把提示词发给图片生成 API，得到图片 URL 或直接下载图片。

scripts/lovart_wrapper.py



图片提示词是怎么生成的？

提示词的质量直接决定图片质量。让我们看一个例子：



风格定位：手绘涂鸦风

项目统一使用"Obsidian 手绘插件风格"，特点是：有机线条、笔记本涂鸦质感、温馨有趣。

提示词模板

CODE

```
STYLE_PREFIX = """
竖版9:16，手绘涂鸦风，
Obsidian手绘插件风格，
有机线条，笔记本涂鸦质感
"""

prompt = f"""{STYLE_PREFIX}
{village_name}村落寻宝活动海报。

主标题：{village_name}村落寻宝
副标题：九宫格探索挑战

元素设计：
- 寻宝主题元素：地图、罗盘、放大镜
- 村庄特色：{features.geography}
- 手绘涂鸦风格

风格要求：
- 竖版9:16比例
- 适合儿童研学活动
- 色彩活泼但不过于花哨"""
```

PLAIN ENGLISH

定义统一的风格前缀

乡村文旅研学生成器

嵌入村庄名称

加入村庄特色元素

明确比例和风格要求

支持的图片类型



课程海报 ×5

每门课程一张海报，展示课程主题



夏令营海报

吸引学生参加夏令营活动



一日游学海报

短途研学体验活动



寻宝卡海报

九宫格寻宝活动宣传



村庄宣传海报

整体形象展示



品牌视觉

Logo、标语、文创产品

 图片服务调用

提示词生成后，需要调用图片服务来生成实际图片。

服务选择策略



自动降级机制：代码会先尝试 Lovart，如果配置了 Lovart API Key 就用它；如果没有配置或调用失败，就自动回退到 MiniMax 的图片生成服务。

图片保存逻辑

CODE

```
def _save_image(self, image_url, poster_path, prompt):
    if image_url:
        resp = httpx.get(image_url, timeout=60)
        poster_path.write_bytes(resp.content)
        return str(poster_path)
    else:
        # 生成失败时，保存提示词作为记录
        poster_path.write_text(f"[失败] {prompt[:80]}")
```

PLAIN ENGLISH

如果拿到了图片 URL

下载图片内容

保存到指定路径

如果失败了，保存提示词作为记录

输出目录结构

- output/
- 龙潭村/
- 龙潭村_研学解决方案.pdf
- 龙潭村_教案1_黄酒酿造.pdf
- 龙潭村_夏令营研学手册.pdf
- 龙潭村_海报_课程1.png
- 龙潭村_Logo.png
- manifest.json

PDF/图片输出，整个流程环环相扣，自动化完成。

在图片生成流程中，什么因素决定了最终图片的质量和风格？

- ☐ A. 图片服务的算法
- ☐ B. 输出文件格式（PNG vs JPG）
- ☐ C. 图片提示词（Prompt）

检查答案

重新尝试